

Programación II

Desarrollo en Java

Clase N° 16:

Genericidad

Profesora Carolina Archuby



Analicemos la problemática con un ejemplo...

Necesitamos crear una clase Caja que pueda almacenar objetos.

Primera solución: Crear una clase Caja que dentro contenga como atributo una colección (ArrayList o cualquier otra).

Desventaja: Debemos definir el tipo de dato de esta colección, con lo cual la caja sólo podrá contener elementos de ese tipo

Segunda solución:

Si queremos que la caja pueda guardar cualquier tipo de objeto, podemos usar la **clase** **Object** como tipo de dato de la colección

```
public class Caja {  
    private List<Object> lista;  
    private int tope;  
  
    public Caja (int tope){  
        this.lista= new ArrayList<>();  
        this.tope= tope;  
    }  
  
    public boolean agregarElemento (Object o){  
        if (tope>lista.size()){  
            lista.add(o) ;  
            return true;  
        }  
        return false;  
    }  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
        Caja miCaja = new Caja(10);  
  
        Caramelo c1 = new Caramelo("Sugus", "menta");  
  
        miCaja.agregarElemento(c1);  
  
        Perro p1 = new Perro();  
        Gato g1 = new Gato();  
  
        miCaja.agregarElemento(p1);  
        miCaja.agregarElemento(g1);  
  
        //Imprimir los caramelos de la caja  
        for (Object o : miCaja.getElementos()) {  
            if (o instanceof Caramelo) {  
                System.out.println(((Caramelo) o).toString());  
            }  
        }  
    }  
}
```



¿Cuál es el problema de usar la clase Object??

=> Al usar Object como tipo de dato, la caja puede contener cualquier cosa... **Queda a cargo del programador realizar las verificaciones necesarias para que el código no falle** □ El uso de Object como tipo de dato es potencialmente inseguro y **no se puede hacer nada para que el programador no cometa un error.**

=> **El error es descubierto en tiempo de ejecución**, al momento de realizar el casteo cuando se lanza una excepción del tipo `ClassCastException`.

La mejor solución
al ejemplo
anterior...

El uso de
GENERICIDAD:

```
public class Caja<T> {  
    private List<T> lista;  
    private int tope;  
  
    public Caja(int tope) {  
        this.lista = new ArrayList<>();  
        this.tope = tope;  
    }  
  
    public boolean agregarElemento(T t){  
        if(tope>lista.size()) {  
            lista.add(t);  
            return true;  
        }  
        return false;  
    }  
}
```

Al instanciar una Caja se define el tipo de dato, y si luego quiere guardarse en ella otro tipo de dato, arrojará error en tiempo de compilación.

```
public class Main {  
  
    public static void main(String[] args) {  
        Caja<Caramelo> miCaja = new Caja(10);  
  
        Caramelo c1 = new Caramelo("Sugus", "menta");  
        miCaja.agregarElemento(c1);  
  
        Perro p1 = new Perro();  
        miCaja.agregarElemento(p1);  
  
        //Imprimir los caramelos de la caja  
        for (Caramelo c : miCaja.getElementos()) {  
            System.out.println(c.toString());  
        }  
    }  
}
```

Error en tiempo de compilación

GENERICIDAD: CONCEPTO

La genericidad es **una serie de técnicas que permiten:**

=> Definir **CLASES, INTERFACES y MÉTODOS**

=> *sin especificar el tipo de datos o parámetros de uno o más de sus miembros,*

=> de forma que puedan trabajar con cualquier tipo de dato (que se definirá y verificará en tiempo de compilación),

=> lo que los hace extremadamente reutilizables.

=> La genericidad no es exclusiva de los lenguajes orientados a objetos, pero es en ellos en los que ha adquirido verdadera importancia y uso

GENERICIDAD - CASOS DE USO

1) CONTENEDORES cuyas operaciones no dependen del tipo de datos almacenado.

Por ejemplo, un ArrayList o una Stack en Java puede almacenar cualquier tipo de objetos (Integer, String, etc, o cualquier objeto definido por el usuario), simplemente especificando el tipo de dato al momento de su creación.

2) ALGORITMOS y operaciones aplicables a cualquier dato, independientemente de su tipo.

Los algoritmos de resolución de numerosos problemas no dependen del tipo de datos que procesa. Por ejemplo, la ordenación.

VENTAJAS DE LA GENERICIDAD

=> **Reutilización de código.** Las clases y métodos genéricos pueden trabajar con cualquier tipo de objeto, lo que los hace extremadamente reutilizables

=> **Evita el casteo de clases , y con ello mejora el rendimiento:** El casteo requiere que la JVM haga chequeo de tipos en tiempo de ejecución, en caso de que sea necesario ejecutar una `ClassCastException`. Con genéricos se evita el chequeo de tipos en tiempo de ejecución.

VENTAJAS DE LA GENERICIDAD

=> **Código más seguro, porque el chequeo de tipos se hace en tiempo de compilación.** La genericidad ayuda a detectar y prevenir errores porque fuerza a verificar el tipo en tiempo de compilación. Reduce la posibilidad de fallos en tiempo de ejecución porque muchos problemas que podrían surgir de malas asignaciones o casteos de tipo inapropiadas se detectan antes de que el programa se ejecute.

=> **Legibilidad y mantenibilidad** del código. Se pueden crear clases e interfaces más limpias y fáciles de entender.

TIPOS GENÉRICOS o PARAMETROS DE TIPO o TIPOS PARAMETRIZADOS

=> Son variables usadas en la definición de CLASES, INTERFACES o MÉTODOS que serán reemplazados por tipos concretos cuando se instancie la clase o se invoque el método.

=> Esto permite que una clase, interfaz o método pueda operar con cualquier tipo de datos, especificado en tiempo de compilación

DECLARACIÓN DE UNA CLASE GENÉRICA

Una clase genérica o parametrizable es
una clase con una o más variables de tipo genérico.

Sintaxis de la clase genérica ->

```
public class MyClassGeneric<T>{  
  
}
```

Donde: **MyClassGeneric** es el nombre de la clase genérica y **T** es un parámetro de tipo que será reemplazado por un tipo real cuando se utilice la clase

Ejemplo de clase genérica

Declaración de la clase:

```
public class ClaseGenerica<T> {  
    T obj;  
  
    public ClaseGenerica( T valor) {  
        obj = valor;  
    }  
  
    public void classType() {  
        System.out.println("El tipo de T es " +  
            obj.getClass().getName());  
    }  
}
```

Implementación:

```
ClaseGenerica<Integer> intObj = new ClaseGenerica<Integer>(88);  
intObj.classType();  
  
ClaseGenerica<String> strObj = new ClaseGenerica<String>("S");  
strObj.classType();  
  
Es esta realmente la salida por pantalla ?  
//El tipo de T es Integer  
//El tipo de T es String
```

Volvamos al ejemplo del inicio de la Caja Genérica para almacenar cualquier tipo de objetos y veamos su implementación

```
public class MiClase <T> {  
    private List<T> lista;  
    public MiClase () {  
        this.lista = new ArrayList<>();  
    }  
    public void imprimirLista() {  
        for (T dato : lista) {  
            System.out.println(dato + " ");  
        }  
        System.out.println();  
    }  
    public void agregarElemento(T elemento){  
        lista.add (elemento);  
    }  
}
```

```
MiClase<Integer>numeros = new MiClase<>();  
numeros.agregarElemento(4) ;  
numeros.imprimirLista();  
MiClase<String> palabras = new MiClase<>();  
palabras.agregarElemento("Hola") ;  
palabras.imprimirLista();
```

No es casual que se parezca tanto a la implementación de ArrayList

```
ArrayList<Integer>numeros = new ArrayList<>();  
numeros.add(1);  
numeros.add(2);  
ArrayList<String>palabras = new ArrayList<>();  
palabras.add ("Hola");  
palabras.add ("mundo");
```

**LAS CLASES e INTERFACES pertenecientes a LAS
COLECCIONES DE JAVA UTILIZAN TIPOS
GENÉRICOS !!!**

TIPOS GENÉRICOS: CONVENCIÓN

Podría designarse cualquier nombre para un genérico en Java, pero para mejorar la legibilidad e interpretación del código existen convenciones para los nombres de estos tipos:

- ❑ **E** → elemento de una colección
- ❑ **K** → clave
- ❑ **N** → número
- ❑ **T** → tipo
- ❑ **V** → valor
- ❑ **S, U, V, etc.** → usado para representar otros tipos.

DECLARACIÓN DE TIPOS GENÉRICOS. RESTRICCIONES.

=> LOS GENÉRICOS EN JAVA SÓLO PUEDEN SER INSTANCIADOS CON CLASES Y NO CON TIPOS PRIMITIVOS (como int, double, etc.).

Por ejemplo, no se puede: `Mapa<String, int>` o `Pila<char>`. Siempre hay que usar las clases envoltorio (Integer, Character, etc.).

=> NO SE PUEDEN CREAR OBJETOS NI ARREGLOS DEL TIPO GENÉRICO.

Por ej: No se puede declarar `T dato = new T( )` ni `T[ ] miArreglo = new T[10]`. Esto no impide declarar variables o argumentos genéricos.

DECLARACIÓN DE TIPOS GENÉRICOS. RESTRICCIONES.

=> Una declaración de tipos genéricos puede tener múltiples tipos parametrizados, separados por comas, dentro de los < >.

Por ejemplo:

```
public class Pair<T1, T2> {  
}
```

=> Todas las invocaciones de clases genéricas son expresiones de una clase. Al instanciar una clase genérica no se crea una nueva clase.

DECLARACIÓN DE TIPOS GENÉRICOS. RESTRICCIONES.

=> No se puede usar el tipo genérico como tipo de un campo estático o en cualquier lugar dentro de un método estático o inicializador estático.

Por ejemplo: no se puede declarar: `private static final T MI_CONSTANTE = new T(`
`).`

=> Dentro de una definición de clases, el tipo genérico puede aparecer en cualquier declaración no estática donde se podría utilizar cualquier tipo de datos concreto.

MÉTODOS GENÉRICOS

=> Un método genérico o plantilla es un método que puede invocarse con una variedad de tipos de objetos diferentes, más allá de los vinculados a la clase en la que están definidos.

=> Un método genérico puede definirse dentro de una clase genérica o dentro de una clase ordinaria, ya que la declaración del tipo es específica para el método y no afecta a la clase entera.

MÉTODOS GENÉRICOS. Forma de utilizarlo:

```
modificadores <T, E, ...> tipo_retorno nombre_metodo (argumentos) {  
    modificador T nombre-Atributo;  
    ...  
}
```

Ejemplo:

```
public <T> void printArray(T[] array) {  
    for (T element : array) {  
        System.out.println(element);  
    }  
}
```

<T> define un parámetro de tipo que es usado por el método printArray. Así, éste método puede ser utilizado con cualquier tipo de arreglo (Integer[], String[], etc)

INTERFACES GENÉRICAS

=> Una interfaz genérica **define un conjunto de métodos que operan en tipos de datos genéricos.**

=> No se especifica con qué tipo de objeto concreto trabajarán los métodos; en su lugar, se utilizan parámetros de tipo que se sustituirán por tipos reales cuando una clase implemente esta interfaz.

Ejemplo de clases ordinarias y genéricas que implementan una interfaz genérica:

```
public interface MyCollection <T> {  
    void add(T objeto);  
    T get();  
    void sort();  
}
```

```
//Implementacion Directa  
public MyClass Implements  
MyCollection<ClaseConcreta> {  
    .... //Implementaciones  
}
```

```
public MyClass <T> Implements MyCollection<T> {  
    void add(T object){  
        ....  
    }  
    T get(){  
        ....  
    }  
    void sort(){  
        ....  
    }  
}
```


LIMITACION AL TIPO GENERICO o PARAMETROS DE TIPO ACOTADO

=> Los parámetros de tipo acotado **permiten limitar los tipos que pueden ser aceptados por un parámetro de tipo en una clase o método genérico**. Es decir: limitar los tipos con los que se puede parametrizar nuestra clase.

=> Esto se hace **especificando una clase o interfaz superior en la jerarquía de la cual el tipo de argumento debe ser una subclase o una implementación**.

=> Para definir un parámetro de tipo acotado, se utiliza la palabra clave **extends** en la declaración del parámetro de tipo (ya sean clases o interfaces).

Limitación al tipo genérico. Ejemplo:

Podríamos querer crear una clase con un método que retorne el mínimo luego de comparar entre 2 objetos. No tendría sentido permitir que se pudiera parametrizar la clase con un tipo de dato que no pueda compararse. Nos interesaría encontrar alguna forma de indicar que el tipo utilizado debe ser un subtipo de Comparable.

```
public static <T extends Comparable> T minimo (T a, T b) {  
  
    if (a == null || b == null )  
        return null;  
  
    if ( a.compareTo( b ) < 0)  
        return a;  
  
    else  
        return b;  
  
}
```

LIMITACION AL TIPO GENERICO

=> La limitación al tipo genérico también puede ser múltiple.

=> En una limitación múltiple, los tipos pueden ser:

a) **sólo una clase**, la cual debe ser la primera, o

b) **interfaces**, las que se deseen, separadas por el ampersand

```
public class <T extends Persona & Comparable> Ejemplo { ... }
```

HERENCIA Y GENERICIDAD

=> Una clase puede heredar de una clase genérica.

=> La nueva clase puede

A) Mantener la genericidad de la clase padre

```
public class CajaSeguridad extends Contenedor
```

B) Restringir la genericidad.

```
public class CajaSeguridad <T extends Valorable> extends Contenedor
```

C) No ser genérica y especificar un tipo concreto.

```
public class CajaSeguridad extends Contenedor<Valorable>
```

COMODINES (wild cards)

- => Es el signo de pregunta ? que se usa en los genéricos para representar “algún tipo desconocido”.
- => Se usa cuando se quiere trabajar con tipos genéricos, pero no importa exactamente qué tipo tienen, o se quiere permitir más de un tipo compatible
- => Hay dos principales formas de comodines:
 - ? extends Type
 - ? super Type

Comodines: ? extends Type

Se usa **para declarar que el tipo desconocido representa un tipo que es una subclase de Type, incluido Type mismo.**

Esto se conoce como un límite superior.

```
public void printList(List<? extends Number> list) {  
    for (Number n : list) {  
        System.out.println(n);  
    }  
}
```

En este ejemplo, el método acepta una lista de objetos que son instancias de Number o sus subclases

Comodines: ? super Type

Se usa **para declarar que el tipo desconocido representa un tipo que es una superclase de Type, incluido Type mismo.**

Esto se conoce como un límite inferior.

```
public void addNumbers(List<? super Integer> list) {  
    list.add(123);  
}
```

En el ejemplo, el método acepta una lista de objetos que son instancias de Integer o de cualquier superclase de Integer (como Number o Object).

```
public class CajaNumerica<T extends Number> {
    private T valor;

    public CajaNumerica(T valor){ this.valor = valor;}

    public T getValor(){ return valor;}
```

```
public void setValor(T valor){ this.valor = valor;}
```

```
// Metodo que usa un comodín
public void mostrarLista(List<? extends Number> lista){
    System.out.println("Contenido de la lista:");
    for (Number n : lista) {
        System.out.println(" - " + n);
    }
    // lista.add(10) ???...
    // NO!! No se puede agregar (tipo desconocido)
}
```

```
// Metodo que usa T directamente (tipo conocido)
public double duplicarValor() {
    return valor.doubleValue() * 2;
}
```

Diferencia entre Comodines (?) y Tipo Genérico (T)

Parte del código	Qué hace	Por qué se usa
class CajaNumerica <T extends Number>	Define un tipo genérico concreto (T)	Podemos usar T internamente, leer y escribir
mostrarLista(List<? extends Number> lista)	Acepta cualquier lista de subtipos de Number	No necesitamos saber el tipo exacto (Integer, Double, etc.)
duplicarValor()	Usa el tipo T conocido	Accede a métodos de Number como doubleValue()

Diferencia entre Comodines (?) y Tipo Genérico (T)

Caso	Se usa en...	Significado	Se puede escribir (add)?	Se puede leer (get)?
T extends Number	Declaración de clase o método genérico	"T es un tipo concreto que hereda de Number"	✓ Sí	✓ Sí
? extends Number	Uso de variables, parámetros o retorno	"Es una lista/colección de algún subtipo de Number"	✗ No	✓ Sí

Idea clave:

- => **T** : **tipo definido** y controlado por vos dentro de la clase o método.
- => **?** : **tipo desconocido**, sólo sabés que cumple cierta restricción (sirve para leer, no para modificar).

Resumen de Comodines Genéricos en Java

Sintaxis	Significado	Dónde se usa	Qué permite hacer	Cuándo conviene usarlo	Ejemplo
<code><?></code>	Tipo desconocido (sin restricción)	Variables, parámetros o retornos	 Leer como <code>Object</code>  No podés agregar nada (salvo <code>null</code>)	Cuando no te importa el tipo genérico, solo necesitás recorrer o imprimir elementos	<pre>List<?> lista = new ArrayList<String>();</pre>
<code>? extends T</code>	Cualquier subtipo de <code>T</code>	Variables, parámetros o retornos	 Leer como <code>T</code> (por ejemplo <code>Number</code>)  No podés agregar (tipo exacto desconocido)	Cuando solo necesitás leer datos (no escribir)	<pre>List<? extends Number> numeros = new ArrayList<Integer>();</pre>
<code>? super T</code>	Cualquier supertipo de <code>T</code>	Variables, parámetros o retornos	 Podés agregar objetos tipo <code>T</code>  Al leer, solo sabés que son <code>Object</code>	Cuando solo necesitás agregar datos (no leerlos)	<pre>List<? super Integer> enteros = new ArrayList<Number>();</pre>